

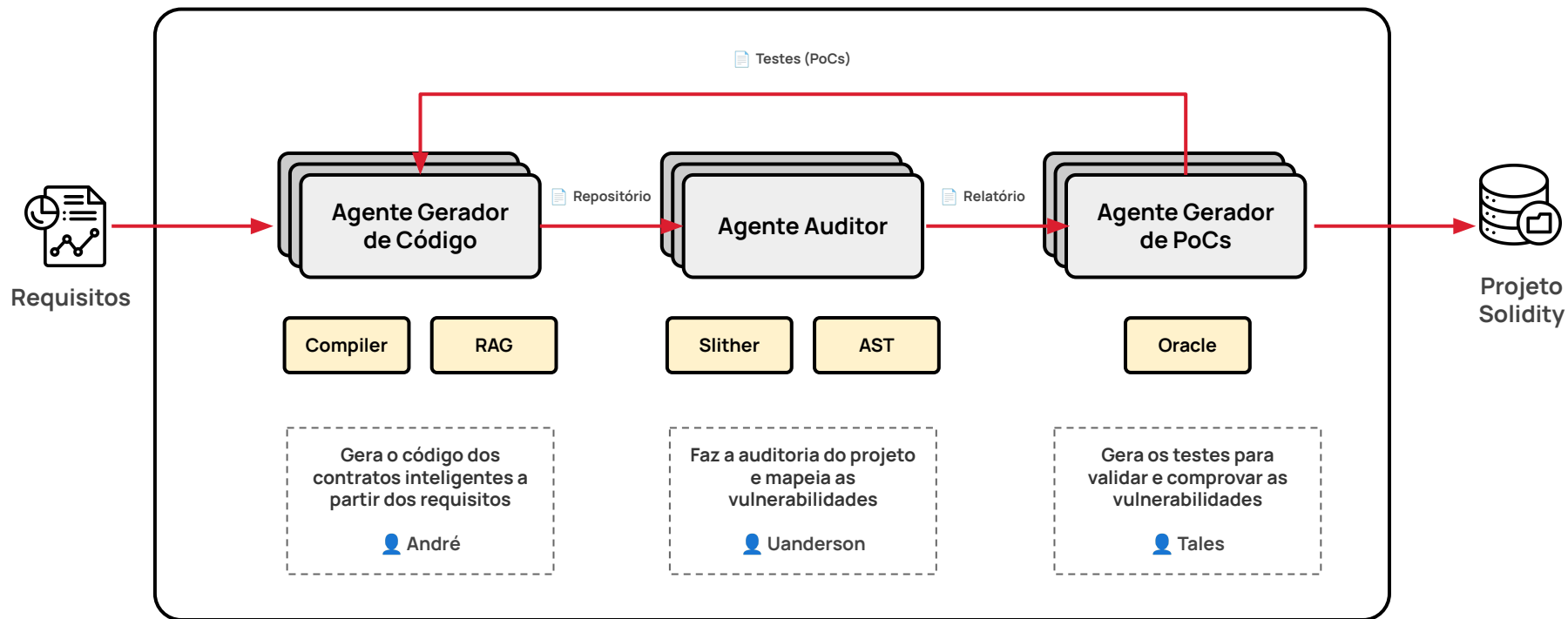
Mestrado

Definição do Projeto

Tópicos Avançados em Linguagens de Programação

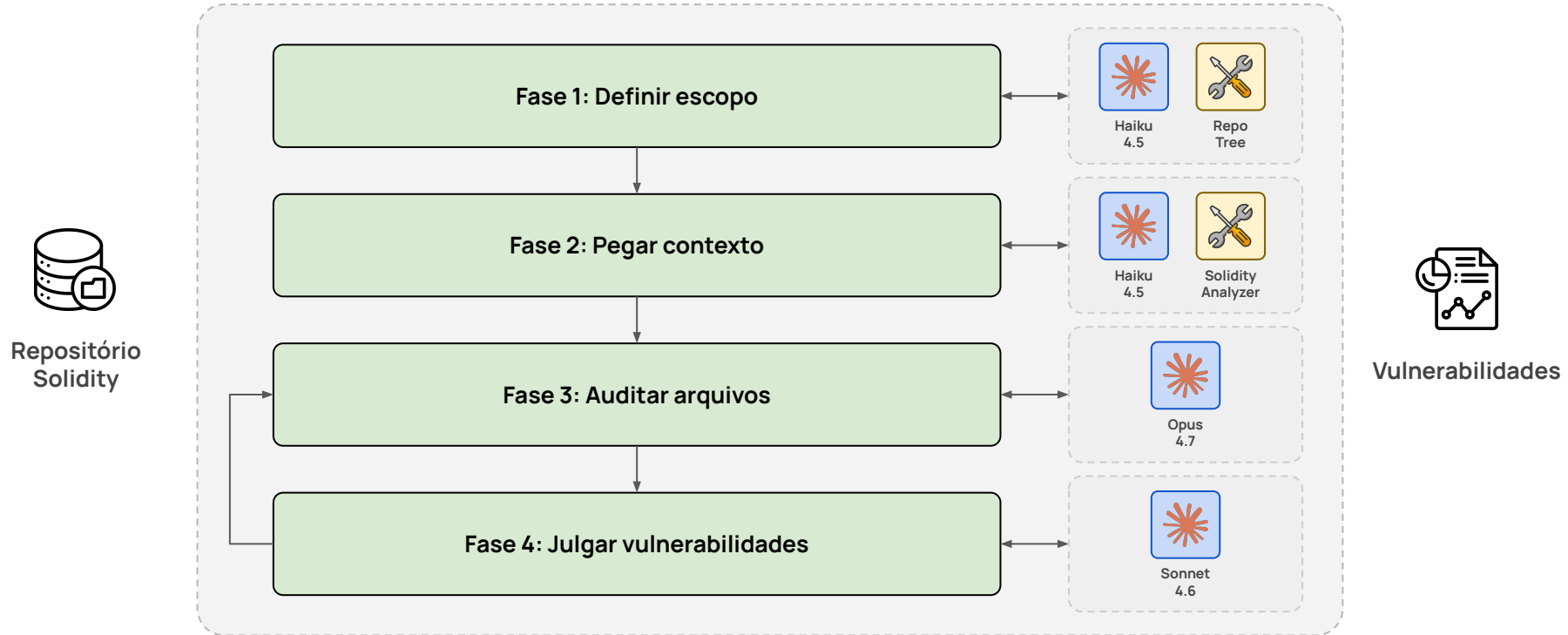
1

Diagrama do Projeto



Auditor

Diagrama do Auditor



Fase 1: Definir escopo

1. Percorre um repositório e marca os arquivos relevantes de código, documentação e configuração
2. Gera a árvore do sistema de arquivos (como contexto para a próxima fase)

```
2025-11-rebatefi-hook/  
├── src/  
│   ├── RebateFiHook.sol [sol]  
│   └── ReFi.sol [sol]  
├── foundry.toml [config]  
└── README.md [doc]
```

3. Rankeia os arquivos de 1-5 pela densidade estimada de vulnerabilidades (importância na descoberta de vulnerabilidades)

Fase 2: Pegar contexto

1. Lê todos os arquivos de documentação
2. Percorre os arquivos de código e gera a análise estrutural de cada um deles
 - a. Essa análise pode ser detalhada ou superficial de acordo com a importância
 - b. Contém herança, storage layout, eventos, modificadores, funções (nome, visibilidade, mutabilidade, parâmetros, retorno, modificadores, descrição, call graph, documentação), chamadas externas, recursão, imports, documentação
3. Compacta toda essa informação em um contexto com tamanho adequado que traz:
 - a. Visão geral dos contratos
 - b. Mapa de estado e armazenamento
 - c. Fluxos principais
 - d. Propriedades e invariantes
 - e. Premissas de design
 - f. Regras de negócio

Fase 2: Pegar contexto

📄 Análise estrutural

ReFiSwapRebateHook · FULL

> path: `2025-11-rebatefi-hook/src/RebateFiHook.sol` | lines: 206 | solc: `0.8.26` | importance: 5/5

Meta

- kind: contract · inherits: [BaseHook, Ownable]
- uses: [`CurrencyLibrary` for `Currency`, `PoolIdLibrary` for `PoolKey`, `LPFeeLibrary` for `uint24`]
- imports: `@openzeppelin/contracts/token/ERC20/IERC20.sol`, `v4-core/types/BeforeSwapDelta.sol`
- docs: @title "ReFiSwapRebateHook - The ReFi Swap Rebate Hook"
- @author "ChoasSR"

Storage

name	type	vis	flags	desc
ReFi	address	public	immutable	-
buyFee	uint24	public	-	-
sellFee	uint24	public	-	-

Events

- `ReFiBought(indexed address buyer, uint256 amount)`
- `ReFiSold(indexed address seller, uint256 amount, uint256 fee)`
- `TokensWithdrawn(indexed address token, indexed address to, uint256 amount)`

Errors

- `ReFiNotInPool()`
- `MustUseDynamicFee()`

External Calls

- `Hooks.Permissions`
- `Currency.unwrap`

Functions

constructor

- visibility: default · mutability: nonpayable · modifiers: [BaseHook, Ownable]
- parameters: `(IPoolManager \_poolManager, address \_ReFi)`
- returns: -
- recurse: no
- state: reads [`ReFi`] · writes [`ReFi`]
- docs: @notice "Initializes the hook with pool manager and ReFi token address" · @param _poolManager: "Address of the Uniswap V4 pool manager" · @param _ReFi: "Address of the ReFi token"

withdrawTokens

- visibility: external · mutability: nonpayable · modifiers: [onlyOwner]
- parameters: `(address token, address to, uint256 amount)`
- returns: -
- calls: [`<expr>.transfer`]
- graph: `withdrawTokens` → IERC20, `withdrawTokens` → TokensWithdrawn
- recurse: no
- docs: @notice "Withdraws tokens from the hook contract" · @dev "Only callable by owner" · @param token: "Address of the token to withdraw" · @param to: "Address to send the tokens to" · @param amount: "Amount of tokens to withdraw"

Fase 2: Pegar contexto

Contexto compactado

Visão geral

RebateFi Hook é um hook Uniswap V4 que implementa estruturas de taxas assimétricas para tokens ReFi (Rebate Finance). O protocolo aplica taxas LP dinâmicas baseadas na direção do swap: zero ou reduzidas para compras (incentivando acumulação) e padrão (0.3%) ou premium para vendas (desestimulando dump e gerando receita). Participantes: owner (controla taxas e saca tokens) e swappers (executam swaps sujeitos às taxas). Ativos protegidos: tokens ReFi acumulados no hook e integridade da lógica de taxas dinâmicas.

Contratos

****ReFi (ERC20, Ownable)**** – Token padrão ERC20 com função mint restrita ao owner. Sem dependências externas críticas além de OpenZeppelin.

****RebateFiHook (BaseHook, Ownable)**** – Hook Uniswap V4 que intercepta swaps e aplica taxas dinâmicas. Herda de BaseHook (v4-periphery) e Ownable. Dependências críticas: IPoolManager (Uniswap V4), IERC20 (para transferências de tokens), endereço imutável do token ReFi.

Estado Crítico

- ``ReFi (address, immutable)`` – Endereço do token ReFi designado. Leitura: qualquer função interna. Escrita: apenas construtor. Impacto: determina qual token recebe taxas assimétricas; manipulação impossível (immutable).
- ``buyFee (uint24, public)`` – Taxa aplicada ao comprar ReFi (em bps ou percentual). Leitura: `_beforeSwap`, `getFeeConfig`. Escrita: `ChangeFee (onlyOwner)`. Impacto: controla incentivo de compra; owner pode definir arbitrariamente.
...

Fluxos Principais

****1. Inicialização de Pool com Hook****
- Chamador invoca `IPoolManager.initialize(PoolKey)` → `_beforeInitialize` valida que ReFi está em uma das currencies → `_afterInitialize` valida que pool usa dynamic fees → pool criado com hook ativo...

Invariantes e Propriedades de Segurança

****Contâteis****

- Saldo de tokens no hook ≥ 0 (não pode ser negativo após transfer).
- Taxas dinâmicas retornadas por `_beforeSwap` devem estar no intervalo válido de uint24.

****De Controle****

- Pool deve usar dynamic fees (validado em `_afterInitialize`); caso contrário, hook não funciona.
- ReFi token deve estar em uma das currencies do pool (validado em `_beforeInitialize`).
- ...

Trust Assumptions

- Owner é confiável e não modificará taxas de forma maliciosa ou arbitrária.
- Token ReFi é ERC20 padrão sem fee-on-transfer, rebase ou hooks maliciosos.
- IPoolManager (Uniswap V4) implementa corretamente a interface e aplica taxas dinâmicas conforme retornado.
- Tokens transferidos via `withdrawTokens` não possuem callbacks maliciosos (reentrância em transfer).
- ...

Regras de Negócio e Restrições de Segurança

- ****Roles****: owner (`ChangeFee`, `withdrawTokens`), swapper (swap via pool).
- ****Modifiers****: onlyOwner em `ChangeFee` e `withdrawTokens`.
- ****Taxas****: `buyFee` e `sellFee` são uint24 sem validação de máximo (podem ser > 100% em bps).
- ****Validações de Pool****: ReFi deve estar em `currency0` ou `currency1`; pool deve usar dynamic fees.
- ****Sem timelock****: `ChangeFee` aplica imediatamente; sem delay entre modificação e efeito.
- ****Sem pausabilidade****: hook não possui função `pause/unpause`.
- ...

Fase 3: Auditar arquivos

1. Recebe o contexto
2. Analisa arquivo por arquivo (em paralelo)
 - a. Título
 - b. Descrição
 - c. Recomendação
 - d. Severidade
 - e. Code Snippet
3. Quando na chamada do loop (reflection), remove os falsos positivos e refina/expande os findings

Fase 4: Julgar vulnerabilidades

1. Recebe o contexto, o arquivo e o finding
2. Revisa finding por finding (em paralelo)
 - a. Revisão
 - b. É falso positivo?
 - c. Recomendação
 - d. Confiança
 - e. Caminhos exploráveis
3. Se alguma das revisões der falso positivo, feedback pra fase anterior

Resultados

```
uanderson@sonson-doomsday:~/mestrado/projeto-talp1$ npm run start
  at async pRetry (file:///home/uanderson/mestrado/projeto-talp1/node_modules/@langchain/core/dist/utils/p-retry/index.js:122:19)
  at async run (/home/uanderson/mestrado/projeto-talp1/node_modules/p-queue/dist/index.js:163:29) {
  status: 400,
  headers: Headers {},
  requestID: 'req_011CbJ8BnKmu7Uvt6dXsujm4',
  error: {
    type: 'error',
    error: {
      type: 'invalid_request_error',
      message: 'Your credit balance is too low to access the Anthropic API. Please go to Plans & Billing to upgrade or purchase credits.'
    },
    request_id: 'req_011CbJ8BnKmu7Uvt6dXsujm4'
  },
  type: 'invalid_request_error',
  pregelTaskId: 'c6124106-c15c-5fd4-a623-53b75c5ca407'
}
```

Avaliação do Auditor



Research Publication

Introducing EVMbench

Making smart contracts safer by evaluating AI agents' ability to detect, patch, and exploit vulnerabilities in blockchain environments.

ou...

- ❑ Curadoria de poucos repositórios
- ❑ Julgar manualmente ou LLM-as-a-judge
- ❑ Comparar com Claude Code

ScaBench: Smart Contract Audit Benchmark

A comprehensive framework for evaluating security analysis tools and AI agents on real-world smart contract vulnerabilities. ScaBench provides curated datasets from recent audits and official tooling for consistent evaluation.

Features

- 🗂️ **Curated Datasets:** Real-world vulnerabilities from Code4rena, Cantina, and Sherlock audits
- 🤖 **Baseline Runner:** LLM-based security analyzer with configurable models
- 📊 **Scoring Tool:** Official Nethermind AuditAgent scoring algorithm
- 📄 **Report Generator:** HTML reports with visualizations and performance metrics
- 📧 **Composable CLI:** End-to-end examples for single or all projects